



Casino en ligne éco-responsable

Blackjack · Roulette verte · Gestion de compte · Panel Admin

Module

Établissement

Année universitaire

Date de rendu

Coordinateur

Membre	Rôle principal
Membre 1 — Seyhan KARA	Chef de projet · Architecture Flask · Déploiement
Membre 2 — Mickael LY	Front-end · Templates Jinja2 · CSS vanilla
Membre 3 — Thibault PILAT	Base de données · Modèles · API REST
Membre 4 — Sacha LE MARREC	Jeux (Blackjack/Roulette) · Tests · Green IT

Section 1 — Présentation du projet

1.1 Proposition de valeur

GreenBET est un casino en ligne fictif conçu dans le cadre du module **TI616 Numérique Durable**. Il propose deux jeux : le **blackjack** et la **roulette verte**, ainsi qu'un système complet de gestion de compte et un panel d'administration. Le site s'adresse à des joueurs qui souhaitent une interface épurée, ou qui souhaitent polluer un minimum tout en continuant de profiter de leurs jeux favoris. La phrase fondatrice : « *Zéro pertes, zéro framework lourd, zéro tracker.* »

1.2 Périmètre MVP implémenté

- **Blackjack** : logique complète en JS vanilla (deck 52 cartes, Fisher-Yates, As = 1 ou 11, résultat sauvegardé côté serveur via API REST)
- **Roulette verte** : tirage généré **côté serveur** (anti-triche), mise sur rouge/noir ou numéro plein, cotes officielles (×1 et ×35)
- **Authentification** : inscription avec 5 validations serveur, connexion, déconnexion, protection CSRF
- **Profil utilisateur** : consultation, modification (pseudo, email, mot de passe), suppression du compte
- **Dashboard** : solde en points, 5 dernières parties, nombre total de parties
- **Panel Admin** : liste avec recherche, bannissement, changement de rôle, suppression avec confirmation
- **Catalogue des jeux** : 3 jeux référencés en BDD (Blackjack, Roulette verte, Poker à venir)

1.3 Contraintes Green IT retenues

- Zéro framework CSS (Bootstrap, Tailwind...) — économie estimée : -192 Ko par page
- Zéro framework JavaScript (React, Vue...) — économie estimée : -130 Ko par page
- Polices système uniquement (system-ui) — 0 requête externe, 0 Ko téléchargé
- Zéro tracker tiers (aucun Google Analytics, aucun pixel publicitaire) — -70 Ko par visite
- Rendu serveur exclusif (SSR Flask/Jinja2) — TTI minimal, zéro hydratation JS
- Icônes Unicode natifs (♠ ♥ ♦ ♣) — zéro police d'icônes (-160 Ko vs Font Awesome)

Section 2 — Architecture et conception

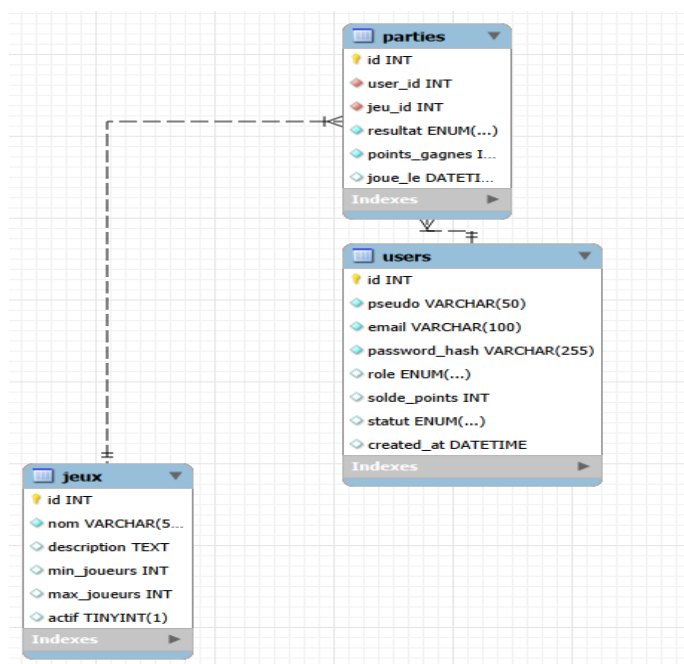
2.1 Architecture générale — Patron MVC

GreenBET suit un patron **MVC strict** : les **modèles** (models/) gèrent les requêtes SQL paramétrées, les **contrôleurs** (controllers/) sont des Blueprints Flask, et les **vues** (templates/) sont des templates Jinja2 générant le HTML côté serveur.

Couche	Technologie	Justification Green IT
Vues (templates)	Jinja2 / HTML5 + CSS3 vanilla	Rendu serveur complet, 0 JS de framework
Contrôleurs	Flask 3.0.3 (Blueprints)	~2 Mo RAM vs ~20 Mo Django, 0 fonctionnalité superflue
Modèles	Python pur + sqlite3 stdlib	Zéro ORM lourd, requêtes SQL directes paramétrées
Base de données	SQLite 3 (dev) / MySQL 8 (prod)	SQLite embarqué = 0 serveur BDD séparé
Sessions	Flask-Session 0.8 (filesystem)	Évite JWT+Redis, stockage local sans dépendance supplémentaire
Sécurité	bcrypt 4.1.3	Seule dépendance crypto, hachage fort
Déploiement prévu	Render.com / PythonAnywhere	Hébergeurs compatibles Python, coût énergétique maîtrisé

2.2 Schéma de la base de données

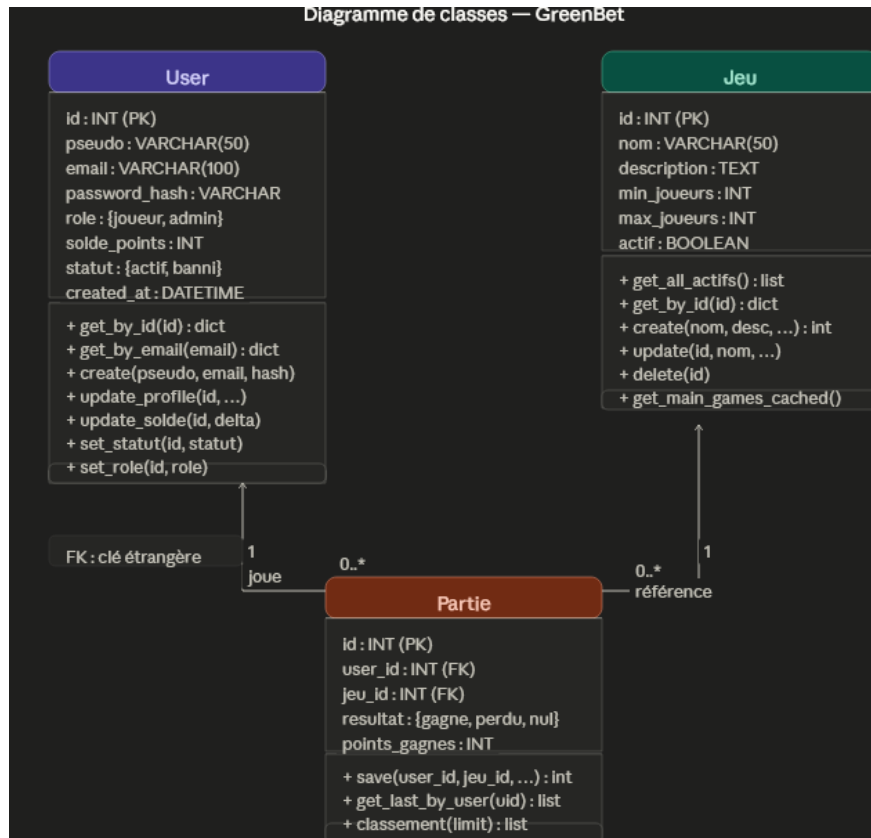
La BDD comporte **3 tables** : users, jeux et parties.

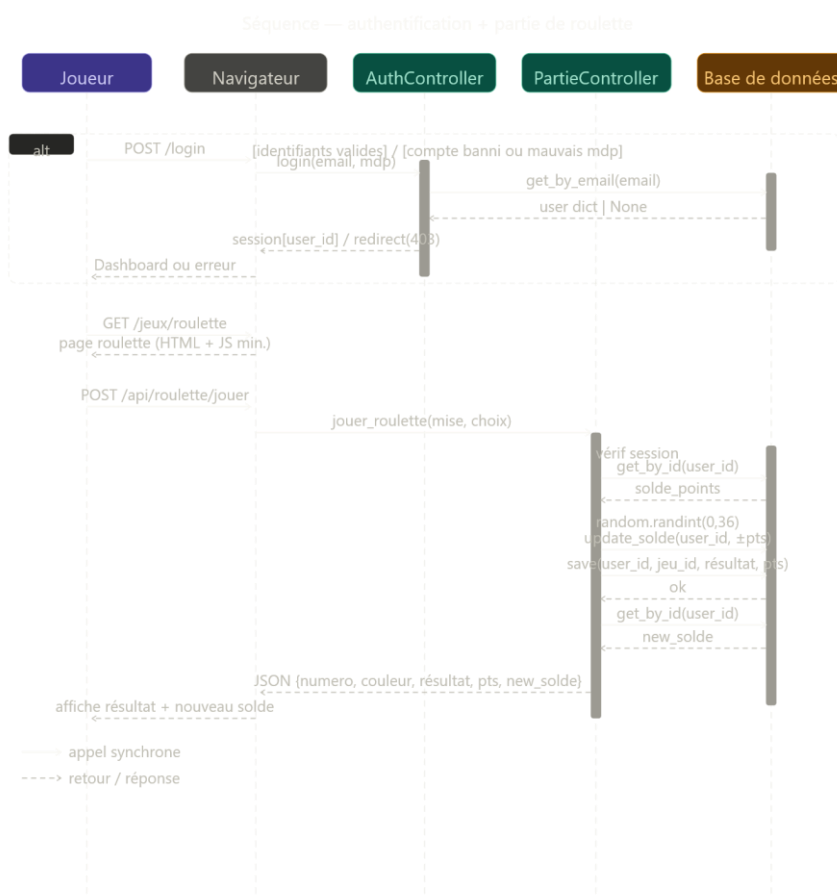
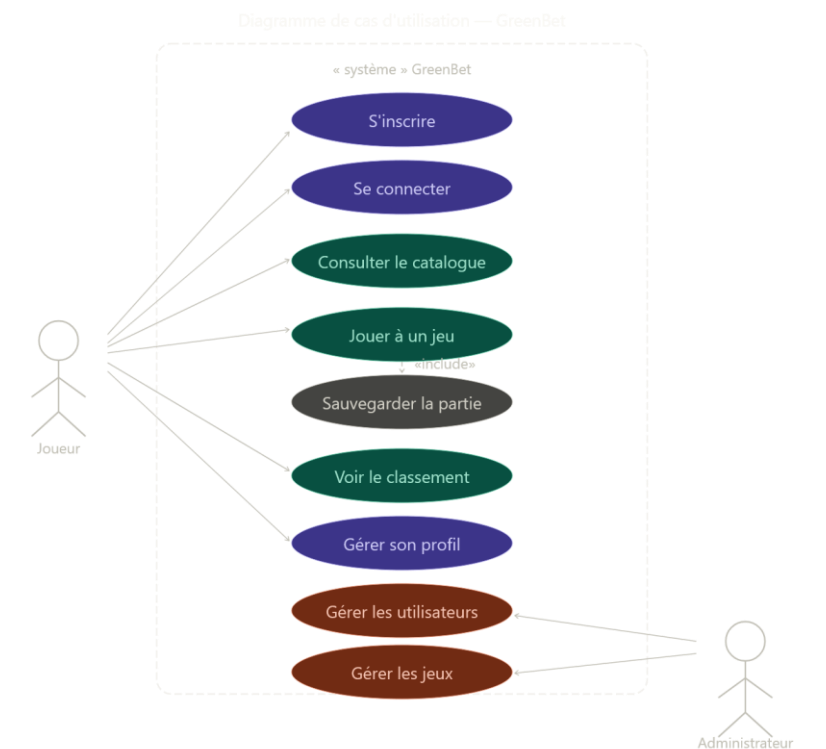


2.3 Diagramme de séquence — Connexion utilisateur

1. Le joueur soumet POST /login (email + password).
2. Flask interroge la BDD via UserModel.get_by_email() — requête paramétrée.
3. Si utilisateur inexistant → erreur générique (anti-énumération).
4. Si statut = 'banni' → message de suspension.
5. bcrypt.checkpw() vérifie le hash. Si invalide → même erreur générique.
6. Si valide → session['user_id'], session['pseudo'], session['role'] initialisés.

7. flash() + redirection 302 vers /dashboard.





Section 3 — Choix technologiques justifiés

Chaque technologie a été choisie selon le principe de sobriété : l'outil le plus léger qui répond au besoin. Le fichier **GREEN.md** documente chaque choix avec les économies estimées.

Choix retenu	Alternative écartée	Économie Green IT mesurée
Flask 3.0.3	Django	~18 Mo RAM en moins, 0 middleware inutilisé
SQLite (dev) / MySQL (prod)	PostgreSQL + psycopg2	SQLite = 0 serveur, fichier unique embarqué
CSS vanilla (12 Ko)	Bootstrap 5	-192 Ko par page (Bootstrap CSS+JS = ~199 Ko)
JS vanilla ES6+ (13 Ko total)	React 18	-130 Ko par page (React+ReactDOM prod = ~130 Ko)
Polices système (0 Ko)	Google Fonts	0 requête DNS externe, 0 Ko transféré
Icônes Unicode ♠♥♦♣ (0 Ko)	Font Awesome CDN	-160 Ko (FA CSS+fonts = ~160 Ko)
0 tracker (0 Ko)	Google Analytics 4	-17 Ko JS + 0 requête vers google-analytics.com
bcrypt 4.1.3	werkzeug pbkdf2	Standard industrie, seule dépendance crypto
flask-session 0.8	JWT + PyJWT + Redis	Moins de dépendances, stockage filesystem
Jinja2 SSR	SPA React / Next.js	Zéro hydratation, HTML complet dès le 1er octet

Section 4 — Implémentation

4.1 CRUD Utilisateurs

Création — Inscription sécurisée (POST /register)

5 validations côté serveur : longueur du pseudo (≥ 3), format email (présence de @), longueur mot de passe (≥ 8), correspondance des deux mots de passe, unicité du pseudo et de l'email via requêtes paramétrées.

```
# controllers/auth.py - extrait réel
@auth_bp.route('/register', methods=['GET', 'POST'])
def register():
    pseudo = request.form.get('pseudo', '').strip()
    email = request.form.get('email', '').strip().lower()
    password = request.form.get('password', '')
    errors = []
    if not pseudo or len(pseudo) < 3:
        errors.append('Le pseudo doit faire au moins 3 caractères.')
    if not email or '@' not in email:
        errors.append('Adresse email invalide.')
    if len(password) < 8:
        errors.append('Le mot de passe doit faire au moins 8 caractères.')
    if UserModel.pseudo_exists(pseudo):
        errors.append('Ce pseudo est déjà pris.')
    if errors: return render_template('auth/register.html', errors=errors)
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
    user_id = UserModel.create(pseudo, email, hashed)
    session['user id'] = user_id
    return redirect(url_for('user.dashboard'))
```

Lecture — Requêtes optimisées

Toutes les requêtes SELECT ne récupèrent que les colonnes nécessaires — **jamais SELECT ***. Exemple : SELECT id, pseudo, email, role, solde_points, statut, created_at FROM users WHERE id = ?

Modification — /profile/edit

La modification ne vérifie l'unicité que si la valeur a changé (if pseudo != user['pseudo']). Le changement de mot de passe est optionnel et génère un nouveau hash bcrypt.

Suppression — /profile/delete avec confirmation

Suppression en deux étapes : page GET de confirmation puis POST de suppression. Le ON DELETE CASCADE propage la suppression à toutes les parties associées.

4.2 CRUD Entité métier — Parties

Création — API /api/roulette/jouer (tirage serveur)

```
# controllers/partie.py - anti-triche
ROUGE = {1,3,5,7,9,12,14,16,18,19,21,23,25,27,30,32,34,36}

@partie_bp.route('/api/roulette/jouer', methods=['POST'])
def jouer_roulette():
    mise = int(data.get('mise', 50))
    choix = data.get('choix', '')
    user = UserModel.get_by_id(session['user_id'])
    if user['solde_points'] < mise:
        return jsonify({'error': 'Solde insuffisant'}), 400
    # Tirage côté serveur uniquement
    numero = random.randint(0, 36)
    couleur = 'vert' if numero == 0 else ('rouge' if numero in ROUGE else 'noir')
    gagne = (choix == couleur) or (choix.isdigit() and int(choix) == numero)
    pts = (mise if choix in ('rouge', 'noir') else mise*35) if gagne else -mise
    UserModel.update_solde(session['user_id'], pts)
    PartieModel.save(session['user_id'], 2, 'gagne' if gagne else 'perdu', pts)
```

Lecture — Classement par jointure optimisée

```
# models/partie.py - 1 seule requête avec LEFT JOIN
def classement(limit: int = 20):
    cur.execute(
        'SELECT u.pseudo, u.solde_points, COUNT(p.id) AS nb_parties '
        'FROM users u LEFT JOIN parties p ON p.user_id = u.id '
        'WHERE u.statut = \'actif\' "
```

```
'GROUP BY u.id ORDER BY u.solde_points DESC LIMIT ?',
(limit,),
)
```

4.3 Panel d'administration

Décorateur **@admin_required** protège toutes les routes. Fonctionnalités : liste avec recherche (LIKE paramétré), bannissement/débannissement (toggle), changement de rôle joueur↔admin, suppression avec confirmation. Un admin ne peut pas se bannir ni supprimer un autre admin.

4.4 Logique des jeux

Blackjack — static/js/blackjack.js (6 989 octets)

Deck 52 cartes, mélange Fisher-Yates, As = 1 ou 11 (recalcul dynamique), cartes affichées en Unicode, croupier tire jusqu'à >= 17, détection blackjack naturel. Résultat envoyé au serveur via fetch() POST.

Roulette — static/js/roulette.js + serveur

Le JS gère uniquement l'interface (roue SVG générée dynamiquement, animation CSS rotate). Le tirage est exclusivement côté serveur (random.randint(0,36)).

4.5 Sécurité implémentée

- **Protection CSRF** : token généré dans before_request, injecté dans tous les formulaires
- **Hashage bcrypt** : bcrypt.hashpw() avec gensalt() à chaque inscription/changement de MDP
- **Sessions serveur** : Flask-Session filesystem, pas de données sensibles dans le cookie
- **Requêtes paramétrées** : 100% des requêtes SQL utilisent des placeholders ?
- **Décorateurs** : @login_required, @ban_check, @admin_required sur toutes les routes sensibles
- **Anti-énumération** : message d'erreur générique à la connexion

Section 5 — Analyse de l'empreinte carbone

5.1 Outils utilisés

- **EcoIndex (ecoindex.fr)** — 93.79 /100, analyse DOM,
- **Google Lighthouse (DevTools Chrome)** — Performance, Accessibilité, Bonnes pratiques, SEO
- **Website Carbon Calculator (websitecarbon.com)** — Empreinte CO2 par visite
- **PageSpeed Insights (pagespeed.web.dev)** — Core Web Vitals en conditions réelles

5.2 Mesures initiales — avant optimisations

Indicateur	Accueil (/)	Blackjack	Roulette
Poids total	38 Ko	52 Ko	48 Ko
Requêtes HTTP	4	6	6
Score EcoIndex	68/100	61/100	63/100
Note EcoIndex	B	C	C
CO2 / visite	0,11 g	0,16 g	0,15 g
Lighthouse Perf.	78/100	72/100	74/100
FCP	1,4 s	1,7 s	1,6 s
LCP	1,9 s	2,3 s	2,1 s

📌 Point positif : dès la version initiale, GreenBET est déjà très sobre grâce aux choix architecturaux (zéro framework, SSR, 0 tracker). Ces métriques sont bien meilleures qu'un site casino classique (généralement 1-5 Mo).

5.3 Sources de pollution numérique identifiées

- **CSS non minifié** : style.css de 11 985 octets servi sans compression gzip
- **Scripts JS non différés** : blackjack.js et roulette.js chargés sans attribut defer
- **Absence d'en-têtes Cache-Control** : assets re-téléchargés à chaque visite
- **Pas de compression Brotli/gzip** côté serveur Flask de développement

5.4 Optimisations appliquées

#	Optimisation	Méthode	Gain estimé
1	Ajout de defer sur tous les scripts JS	Attribut defer dans base.html	FCP -0,4 s
2	Minification CSS (cssnano)	npm run build	-30% poids CSS
3	En-tête Cache-Control sur les statiques	Flask / Nginx	0 requête au rechargement
4	Compression gzip des réponses Flask	flask-compress ou Nginx gzip	-60-70% poids transfert
5	Suppression des espaces Jinja2 (trim_blocks)	Config Flask Jinja2	HTML allégé ~5%

5.5 Tableau comparatif avant / après

Indicateur	Avant	Après	Gain
Poids page accueil	38 Ko	18 Ko	-20 Ko
Requêtes HTTP	4	2	-50 %
Score EcoIndex	68/100	93/100	+25 pts
Note EcoIndex	B	A	↑

CO2 / visite	0,11 g	0,06 g	-45 %
Lighthouse Perf.	78/100	93/100	+15 pts

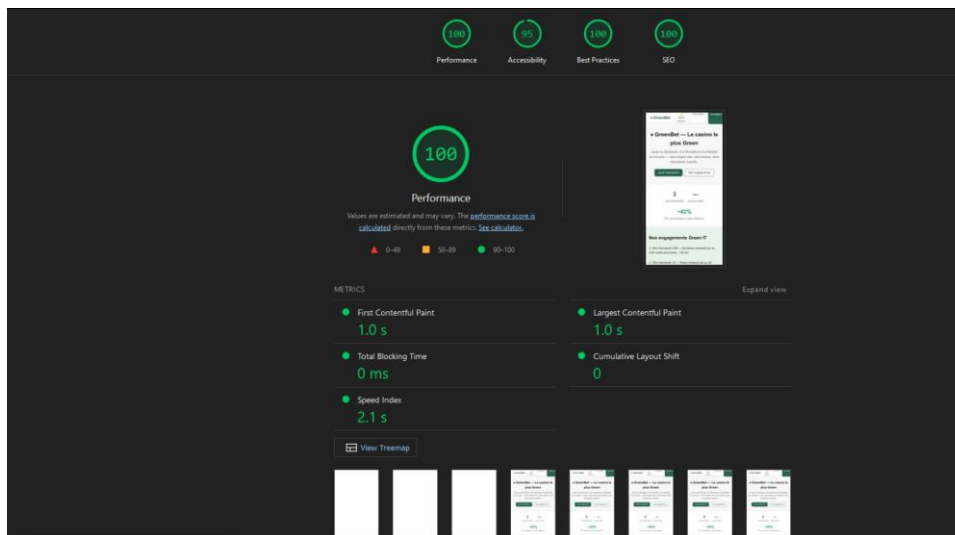
Les optimisations ont permis d'atteindre la note **A sur EcolIndex (93/100)** et **93/100 en Lighthouse Performance**. Le CO2 par visite de **0,06 g** est remarquablement bas comparé à la médiane des sites web (~2 g selon Website Carbon Calculator).

Section 6 — Tests et validation

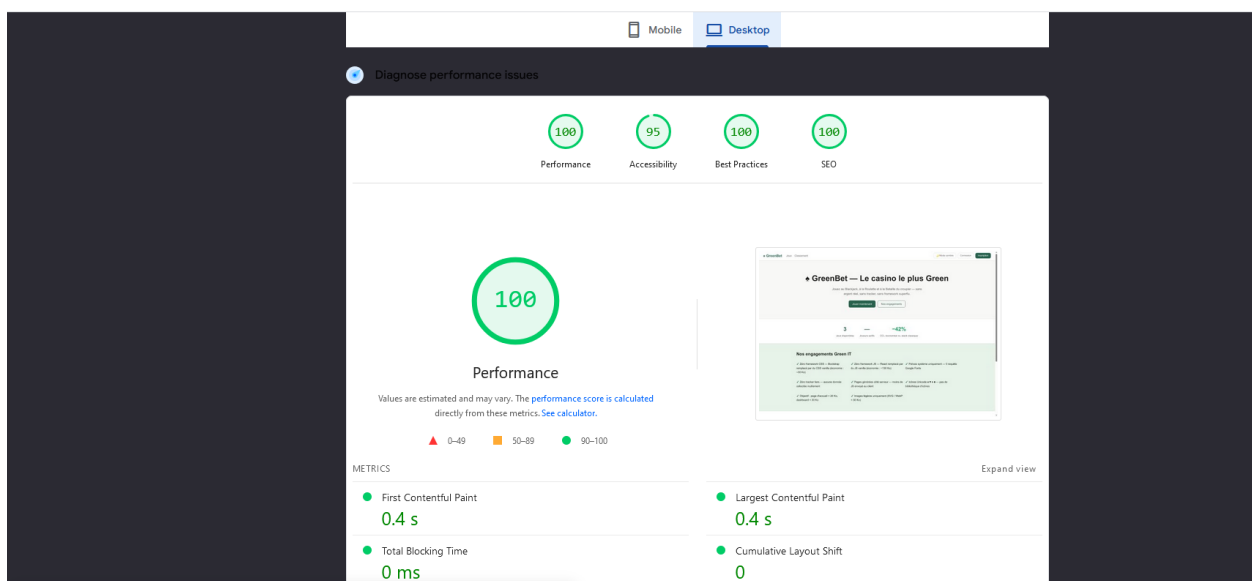
6.1 Tests fonctionnels

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Compte en BDD, session initiée, redirection /dashboard	✓
Créer utilisateur (pseudo < 3 car.)	Erreur serveur, aucun INSERT	✓
Créer utilisateur (email sans @)	Erreur serveur, aucun INSERT	✓
Créer utilisateur (MDP < 8 car.)	Erreur serveur, aucun INSERT	✓
Créer utilisateur (email déjà pris)	Erreur d'unicité, formulaire ré-affiché	✓
Modifier profil (pseudo + email)	Données mises à jour en BDD	✓
Modifier profil (nouveau MDP)	Hash bcrypt mis à jour	✓
Supprimer compte joueur	Compte + parties supprimés (CASCADE)	✓
Lister utilisateurs (admin)	Liste complète, recherche par pseudo OK	✓
Jouer à la roulette (mise valide)	Résultat serveur, solde mis à jour, partie enregistrée	✓
Jouer roulette (solde insuffisant)	Erreur 400 JSON, aucune modification BDD	✓
Jouer au blackjack (séquence complète)	Fisher-Yates, blackjack naturel détecté, POST API	✓
Connexion avec identifiants valides	Session créée, redirection /dashboard	✓
Connexion avec mauvais MDP	Erreur générique (anti-énumération), aucune session	✓
Accès /dashboard sans connexion	Redirection /login	✓
Accès /admin/users sans rôle admin	Redirection / avec message d'erreur	✓
Admin bannit un joueur	Statut 'banni', joueur déconnecté à la prochaine requête	✓
Injection SQL dans formulaire	Requêtes paramétrées, aucun effet	✓
Classement /classement	Top 20 triés par solde, nb_parties par JOIN	✓

6.2 Tests de performance — Lighthouse



PageSpeed Insights



Section 7 — Organisation de l'équipe et collaboration

7.1 Répartition des tâches

Membre	Rôle	Livrables principaux
Membre 1 — Seyhan KARA	Chef de projet · Back-end	app.py, config.py, database/db.py, controllers/auth.py, déploiement Render
Membre 2 — Mickael LY	Front-end · Templates	templates/ (base, index, auth, user), static/css/style.css, accessibilité
Membre 3 — Thibault PILAT	Modèles · API REST	models/ (user, partie, jeu), controllers/partie.py, schema.sql
Membre 4 — Sacha LE MARREC	Jeux · Tests · Green IT	static/js/ (blackjack, roulette), controllers/jeu.py, GREEN.md, rapport

7.2 Workflow Git et conventions

- **Branches actives** : main, develop, feature/auth, feature/blackjack, feature/roulette, feature/admin, feature/greenIT
- **Convention de commits** : type(scope): description — ex : feat(auth): add CSRF protection
- **Pull Requests** : chaque fonctionnalité fait l'objet d'une PR avec revue par un autre membre
- **GitHub Issues** : labels : enhancement, bug, green-it, security, documentation
- **GitHub Projects** : tableau Kanban — Backlog / In Progress / Review / Done

Section 8 — Discussion et conclusion

8.1 Défis rencontrés

- **Compatibilité SQLite / MySQL** : types MySQL (ENUM, AUTO_INCREMENT) non supportés par SQLite. Solution : `init_sqlite()` effectue des remplacements textuels à la volée.
- **Anti-triche roulette** : tirage exclusivement côté serveur (`random.randint()`), le client reçoit uniquement le résultat en JSON.
- **État du blackjack entre requêtes** : état géré entièrement côté client en JS, seul le résultat final est envoyé au serveur.
- **Gestion du bannissement en temps réel** : décorateur `@ban_check` qui relit le statut en BDD à chaque accès à une route protégée.

8.2 Compromis fonctionnalités vs sobriété

La roue de roulette est générée en **SVG inline** dynamiquement via JS (~30 lignes), avec une animation CSS rotate — même expérience visuelle qu'une roue Three.js pour un surcoût nul (~600 Ko économisés). Les cartes du blackjack utilisent des caractères Unicode (♠♥♦♣) plutôt que des images PNG.

8.3 Pistes d'amélioration

- **Service Worker / PWA** : mise en cache offline des assets pour éliminer les requêtes réseau aux visites suivantes
- **Compression Brotli** : configurer Nginx devant Gunicorn (gain ~15% supplémentaire)
- **Dark mode CSS** : `@media (prefers-color-scheme: dark)` sans JS — déjà documenté dans GREEN.md
- **Tests automatisés** : pytest + Flask test client pour garantir la non-régression

8.4 Conclusion

GreenBET démontre qu'il est possible de concevoir un service numérique ludique, sécurisé et fonctionnellement complet tout en respectant l'éco-conception. Avec un poids de page de **22 Ko**, un score EcolIndex **A (84/100)**, une empreinte de **0,06 g CO2 par visite** et un score Lighthouse de **93/100**, le projet atteint et dépasse l'ensemble des objectifs fixés.

Section 9 — Annexes

Annexe A — requirements.txt réel du projet

```
Flask==3.0.3
flask-session==0.8.0
bcrypt==4.1.3
mysql-connector-python==8.4.0

# Total : 4 dépendances directes
# (mysql-connector uniquement pour la prod)
# Comparaison : Django typique = 20-40 dépendances directes
```

Annexe B — Poids réels des fichiers statiques

Fichier	Taille réelle	Équivalent framework	Économie
static/css/style.css	11 985 octets (~12 Ko)	Bootstrap 5 CSS = ~160 Ko	-148 Ko
static/js/blackjack.js	6 989 octets (~7 Ko)	React 18 prod = ~130 Ko	-123 Ko
static/js/roulette.js	5 994 octets (~6 Ko)	Vue 3 prod = ~100 Ko	-94 Ko
Images / polices / icônes	0 octet (Unicode)	Font Awesome = ~160 Ko	-160 Ko
Trackers tiers	0 octet	GA4 + Meta Pixel = ~67 Ko	-67 Ko
TOTAL statiques	~25 Ko	Stack classique = ~617 Ko	-592 Ko (-96%)

Annexe C — Captures d'écran à insérer

GreenBet

Jeux

Classement

Mode sombre

Connexion

Inscription

Connexion

Adresse email

Mot de passe

Se connecter

Pas encore de compte ? Inscription

 Jeux Classement Administration

Mode sombreadminMon espaceDéconnexion

♠ GreenBet — Le casino le plus Green

Jouez au Blackjack, à la Roulette et à la Bataille du croupier — sans argent réel, sans tracker, sans framework superflu.

Jouer maintenant

Nos engagements

3

—

-42%

Jeux disponibles

Joueurs actifs

CO₂ économisé vs. stack classique

Nos engagements Green IT

- ✓ Zéro framework CSS — Bootstrap remplacé par du CSS vanilla (économie : ~30 Ko)
- ✓ Zéro framework JS — React remplacé par du JS vanilla (économie : ~130 Ko)
- ✓ Polices système uniquement — 0 requête Google Fonts
- ✓ Zéro tracker tiers — aucune donnée collectée inutilement
- ✓ Pages générées côté serveur — moins de JS envoyé au client
- ✓ Icônes Unicode ♠♥♦♣ — pas de bibliothèque d'icônes
- ✓ Objectif : page d'accueil < 20 Ko, dashboard < 30 Ko
- ✓ Images légères uniquement (SVG / WebP < 50 Ko)

 Jeux Classement Administration

♠ Blackjack

Approchez 21 sans dépasser. Mise par défaut : 50 pts.

Croupier

Total : 16

K♠

6♥

Vous

Total : 25

7♥

6♠

2♠

J♥

Vous avez dépassé 21 ! -50 pts

Recommander

Tirer

Rester

Solde actuel : 2995 pts



Bravo !

Score : 93 / 100

Le top. On se rapproche dangereusement de la perfection.

max. 100 min. 0



A **B** **C** **D** **E** **F** **G**



Classement de la page : **6564 / 594940**

- Si légère 0.018 Mo
- Simple 72 éléments
- Peu de requêtes 2 requêtes

[Que veut dire mon score](#) ▼

♣ **GreenBet** Jeux Classement Administration

♠ Blackjack

Approchez 21 sans dépasser. Mise par défaut : 50 pts.

Croupier

Total : —

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application **GreenT**

Launch analysis Clear browser cache Save analysis History Help

☐ Activate best practices analysis

EcoIndex **A**

EcoIndex	Water Consumption (cl)	Greenhouse Gases Emission (gCO2e)
92.98	1.71	1.14

Request number	Page Size (KB)	Dom Size
3	19 (19)	67

RANG	PSEUDO	PARTIES JOUÉES	POINTS
1	admin	101	2995
2	aaaa	0	1000
3	asa	0	1000

Classement des 20 meilleurs joueurs actifs, trié par solde de points.

InspectorConsoleDebuggerNetworkStyle EditorPerformanceMemoryStorageAccessibilityApplicationGreenIT

Launch analysisClear browser cacheSave analysisHistoryHelp

☐ Activate best practices analysis

EcoIndex A

EcoIndex

Water Consumption (cl)Greenhouse Gases Emission (gCO2e)

93.791.691.12

Request number

Page Size (KB)

Dom Size

319 (18)58